

SQL

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

- ★ 1. Introduction to SQL
- ★ 2. The SELECT Statement
- ★ 3. The WHERE Clause
- ★ 4. INSERT, UPDATE, and DELETE
- 5. Sorting - ORDER BY
- 6. Filtering - DISTINCT and LIMIT
- ★ 7. Aggregate Functions (COUNT, SUM, AVG, MAX, MIN)
- ★ 8. GROUP BY and HAVING
- ★ 9. JOINS (INNER, LEFT, RIGHT, FULL)
- ★ 10. Subqueries
- ★ 11. CREATE TABLE and Data Types
- 12. ALTER TABLE
- ★ 13. Primary Key and Foreign Key
- 14. Indexes
- 15. Views
- ★ 16. Transactions (COMMIT and ROLLBACK)

★ 1. Introduction to SQL *[IMPORTANT TOPIC]*

Explanation:

SQL (Structured Query Language) is the standard language used to communicate with a relational database — to store, retrieve, update, and delete data organized in tables (rows and columns).

Almost every major database system (MySQL, PostgreSQL, SQL Server, Oracle) understands SQL, with only small differences between them.

Key Points:

- Used to manage data stored in tables.
- Works the same basic way across most database systems.
- SQL keywords are usually written in UPPERCASE by convention (though not required).

Example Code:

```
SELECT * FROM students;
```

Output:

Returns every row and every column from the 'students' table.

★ 2. The SELECT Statement *[IMPORTANT TOPIC]*

Explanation:

SELECT is the most commonly used SQL command — it is used to RETRIEVE (read) data from one or more tables.

Example Code:

```
SELECT name, age FROM students;
```

Output:

name	age
Pravendra	21
Aman	22

★ 3. The WHERE Clause *[IMPORTANT TOPIC]*

Explanation:

WHERE is used to FILTER data, so we only get the rows that match a specific condition, instead of every row in the table.

Example Code:

```
SELECT * FROM students WHERE age > 21;
```

Output:

Returns only the students whose age is greater than 21.

★ 4. INSERT, UPDATE, and DELETE [IMPORTANT TOPIC]

Explanation:

These three commands let us CHANGE the data inside a table. INSERT adds a new row. UPDATE changes an existing row. DELETE removes a row completely.

Example Code:

```
-- Add a new student
INSERT INTO students (name, age) VALUES ('Riya', 20);

-- Update an existing student's age
UPDATE students SET age = 23 WHERE name = 'Aman';

-- Delete a student
DELETE FROM students WHERE name = 'Riya';
```

Output:

```
1 row inserted.
1 row updated.
1 row deleted.
```

 *Note: ALWAYS use a WHERE clause with UPDATE and DELETE — forgetting it will change or delete EVERY row in the table!*

5. Sorting - ORDER BY

Explanation:

ORDER BY is used to sort the results of a query, either in ascending order (ASC, the default) or descending order (DESC).

Example Code:

```
SELECT name, age FROM students ORDER BY age DESC;
```

Output:

name	age
Aman	23
Pravendra	21

6. Filtering - DISTINCT and LIMIT

Explanation:

DISTINCT removes duplicate values from the result. LIMIT restricts how many rows are returned — useful for previewing large tables.

Example Code:

```
SELECT DISTINCT city FROM students;  
  
SELECT * FROM students LIMIT 5;
```

Output:

Returns each unique city only once.
Returns only the first 5 rows of the table.

★ 7. Aggregate Functions (COUNT, SUM, AVG, MAX, MIN)

[IMPORTANT TOPIC]

Explanation:

Aggregate functions perform a calculation across MANY rows and return a single summary result — very useful for reports and statistics.

Key Points:

- COUNT() -> counts the number of rows
- SUM() -> adds up all values
- AVG() -> calculates the average
- MAX() / MIN() -> finds the largest / smallest value

Example Code:

```
SELECT COUNT(*) FROM students;  
SELECT AVG(age) FROM students;
```

Output:

```
COUNT(*)
```

```
-----
```

```
3
```

```
AVG(age)
```

```
-----
```

```
22.0
```

★ 8. GROUP BY and HAVING [IMPORTANT TOPIC]

Explanation:

GROUP BY groups rows that share the same value in a column, usually combined with an aggregate function, like finding the average marks PER CLASS instead of for everyone at once. HAVING is used to filter these groups, similar to how WHERE filters normal rows.

Example Code:

```
SELECT class, AVG(marks) FROM students
GROUP BY class
HAVING AVG(marks) > 70;
```

Output:

class	AVG(marks)
10-A	78.5

 *Note: WHERE filters rows BEFORE grouping; HAVING filters groups AFTER grouping.*

★ 9. JOINS (INNER, LEFT, RIGHT, FULL) [IMPORTANT TOPIC]

Explanation:

A JOIN combines rows from TWO OR MORE tables based on a related column between them — one of the most important and commonly used SQL concepts.

Key Points:

- INNER JOIN -> returns only matching rows from both tables.
- LEFT JOIN -> returns ALL rows from the left table, plus matches from the right (or empty if no match).
- RIGHT JOIN -> returns ALL rows from the right table, plus matches from the left.
- FULL JOIN -> returns ALL rows from both tables, matched where possible.

Example Code:

```
SELECT students.name, courses.course_name
FROM students
INNER JOIN courses ON students.course_id = courses.id;
```

Output:

name	course_name
Pravendra	Python
Aman	Java

★ 10. Subqueries *[IMPORTANT TOPIC]*

Explanation:

A subquery is a SELECT query written INSIDE another SQL query — used when we need the result of one query to help build another.

Example Code:

```
SELECT name FROM students  
WHERE age > (SELECT AVG(age) FROM students);
```

Output:

Returns the names of all students older than the average age.

★ 11. CREATE TABLE and Data Types *[IMPORTANT TOPIC]*

Explanation:

CREATE TABLE is used to build a new table, defining each column's name and data type — the structure that will hold future data.

Example Code:

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    age INT,  
    joined_date DATE  
);
```

Output:

Table 'students' created successfully.

12. ALTER TABLE

Explanation:

ALTER TABLE is used to CHANGE the structure of an already-existing table — like adding a new column, without needing to recreate the whole table.

Example Code:

```
ALTER TABLE students ADD COLUMN email VARCHAR(100);
```

Output:

Table 'students' altered – new column 'email' added.

★ 13. Primary Key and Foreign Key *[IMPORTANT TOPIC]*

Explanation:

A PRIMARY KEY uniquely identifies each row in a table — no two rows can have the same value. A FOREIGN KEY is a column that links to the PRIMARY KEY of ANOTHER table, creating a relationship between the two tables.

Example Code:

```
CREATE TABLE courses (  
    id INT PRIMARY KEY,  
    course_name VARCHAR(50)  
);  
  
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course_id INT,  
    FOREIGN KEY (course_id) REFERENCES courses(id)  
);
```

Output:

Both tables created, now properly linked through course_id.

14. Indexes

Explanation:

An index is a special lookup structure that makes SEARCHING a table much faster, similar to an index at the back of a book. Indexes speed up SELECT queries, but slightly slow down INSERT/UPDATE, since the index must also be updated.

Example Code:

```
CREATE INDEX idx_name ON students(name);
```

Output:

Index 'idx_name' created – searches by name are now much faster.

15. Views

Explanation:

A View is a saved, virtual table based on a SQL query. It doesn't store data itself, but lets us reuse a complex query easily, just by 'selecting' from the view like a normal table.

Example Code:

```
CREATE VIEW adult_students AS  
SELECT name, age FROM students WHERE age >= 18;  
  
SELECT * FROM adult_students;
```

Output:

Behaves exactly like a table, always showing current students aged 18 or older.

★ 16. Transactions (COMMIT and ROLLBACK) [IMPORTANT TOPIC]

Explanation:

A transaction groups several SQL commands together, so they either ALL succeed together, or NONE of them happen at all — protecting data from being left in a broken, half-completed state. COMMIT saves the changes permanently; ROLLBACK undoes them.

Example Code:

```
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE name = 'A';
UPDATE accounts SET balance = balance + 500 WHERE name = 'B';

COMMIT;    -- saves both changes permanently
-- ROLLBACK;    -- would undo both changes instead
```

Output:

Both balance updates are saved together, or not at all.

 *Note: Transactions are essential for tasks like money transfers, where a half-finished operation could cause serious problems.*